

Go Crypto Wallet — Full Project Roadmap (Concept → Production)

Repo: `go-crypto-wallet` **Goal of this document:** take you from an empty repo to a deployed, observable system — and along the way, actually teach you crypto/Web3 fundamentals and distributed systems, not just leave you with working code you can't explain.

How to use this

Work top to bottom. Each phase has a goal, the concepts it teaches, concrete tasks, and a "you're done when" checklist. Don't skip to later phases for the interesting parts — the boring correctness work in Phase 1 is what makes Phase 2 onward trustworthy.

Where a phase says "read first," that's a short, specific pointer — not homework. Read just enough to unblock the tasks, then come back and build.

Phase 0 — Foundations (read alongside Phase 1, don't front-load it all)

You don't need to master this before writing code — you need just enough to know what question you're answering at each step.

Crypto/Web3 basics to pick up:

- What a private key, public key, and address actually are (a wallet doesn't "hold" coins — it holds a key that can sign transactions)
- What a transaction is on a chain like Ethereum, and what "confirmation" and "block" mean
- Why reorgs happen and why they matter for anyone crediting a deposit
- *Read first:* skim "Mastering Bitcoin" (Antonopoulos) ch. 1–4 for the conceptual model — it explains keys/addresses more clearly than most Ethereum-specific docs, and the concepts transfer directly.

Distributed systems basics to pick up:

- At-least-once vs exactly-once delivery (you already hit this with idempotency)
- Replication and partitioning, at a high level
- Consensus, at a high level (you don't need to implement Raft — you need to recognize when you're relying on it)
- *Read first:* "Designing Data-Intensive Applications" (Kleppmann) ch. 1 and ch. 5 intro — save the rest for Phase 3, where it becomes directly relevant.

Phase 1 — Core Ledger (in progress)

Goal: a correct, concurrency-safe, idempotent internal accounting system. No blockchain yet — this phase is entirely about proving you can move money correctly, which is the foundation everything else sits on.

What you already have: `schema.sql`, `transfer_test.go` (the spec), and stubbed `transfer.go` / `retry.go`.

Tasks:

- ☐ Implement `withRetry` in `retry.go`
- ☐ Implement `Transfer` in `transfer.go` — atomic idempotency claim, ordered row locks, balance check, double-entry ledger write
- ☐ Wire `setupTestDB` to a real local Postgres (Docker is easiest: `docker run -e POSTGRES_PASSWORD=postgres -p 5432:5432 postgres`)
- ☐ Get both tests in `transfer_test.go` passing
- ☐ Run `go test -race ./...` and confirm no race warnings

Concepts this teaches: idempotency, row-level locking and deadlock avoidance, database transactions, double-entry accounting.

Done when: `go test -race ./...` passes, and you can explain out loud *why* the lock order matters — not just that it does.

Phase 2 — Real Web3 Integration

Goal: turn the ledger into an actual crypto wallet by connecting it to a real chain.

Tasks:

- ☐ Add `go-ethereum`'s client library (`go-ethereum/ethclient`) to the project
- ☐ Generate a real key pair and derive an address (use a Sepolia or other public testnet — never mainnet for this project)
- ☐ Fund the test address from a testnet faucet
- ☐ Build a **deposit watcher**: poll or subscribe for incoming transactions to your address, and once a transaction has enough confirmations (pick a number, e.g. 12, and write down *why* that number), credit the corresponding internal ledger balance via `Transfer`
- ☐ Build a **withdrawal path**: given an internal debit, construct, sign, and broadcast an actual testnet transaction

Concepts this teaches: public-key cryptography in practice, how blocks/confirmations work, reorg risk (what happens if you credit a deposit and then the block it was in gets

reorg'd out — this is a real, well-known crypto-systems bug class), transaction signing.

Done when: you can send testnet ETH to your generated address from an external wallet (e.g. MetaMask) and watch your internal ledger balance update automatically, and you can withdraw back out.

Phase 3 — Distributed Systems Hardening

Goal: stop assuming a single process and a perfectly reliable network. This is where the DDIA reading becomes concrete instead of abstract.

Tasks:

- ☐ Decouple the deposit watcher from processing using a queue (NATS or Kafka both work — NATS is lighter to run locally) — the watcher publishes "new deposit seen" events, a separate consumer credits the ledger
- ☐ Make the consumer idempotent against redelivery (queues give at-least-once delivery — you already know how to handle this from Phase 1)
- ☐ Introduce a second instance of the deposit-watcher consumer and add a distributed lock (Redis-based, e.g. Redlock, or a Postgres advisory lock) so only one instance processes a given block range at a time
- ☐ Write a chaos test: kill the consumer mid-processing (literally `kill -9` it) and confirm no deposit is lost or double-credited on restart

Concepts this teaches: message queues and delivery guarantees, distributed locking beyond a single database, designing for partial failure — the actual substance of "distributed systems," not just the vocabulary.

Done when: you can kill any single component mid-operation and the system recovers without data loss or duplication.

Phase 4 — API & Service Layer

Goal: expose the wallet as a real service, not just a library.

Tasks:

- ☐ Build a REST (or gRPC, if you want the practice) API: create account, get balance, initiate transfer, initiate withdrawal, get transaction history
- ☐ Add auth (API keys are enough for a portfolio project — don't over-engineer this part)
- ☐ Add request validation and rate limiting
- ☐ Add API-level idempotency-key support (clients supply their own key in a header — this is how Stripe's API works, and it's worth mirroring)

Concepts this teaches: API design for financial operations, why idempotency needs to be a client-facing contract, not just an internal implementation detail.

Phase 5 — Observability

Goal: you should be able to answer "what is this system doing right now, and why did that transfer fail?" without reading code.

Tasks:

- ☐ Structured logging (use `slog`, Go's standard structured logger) — every log line should carry a request ID / idempotency key so you can trace one transfer end to end
- ☐ Metrics via Prometheus client library: transfer count, transfer latency histogram, failed-transfer counter by reason, queue lag
- ☐ Distributed tracing via OpenTelemetry — trace a request from API → queue → consumer → DB, so you can see where time actually goes
- ☐ A Grafana dashboard built on top of the above (even a simple one)
- ☐ One real alert: e.g. "queue consumer lag > 60s" wired to fire (a webhook to your own Slack/email is enough)

Concepts this teaches: the three pillars of observability (logs, metrics, traces) and — more importantly — the discipline of instrumenting as you build, not bolting it on after.

Phase 6 — Deployment

Goal: a real, running deployment, not just code that works on your laptop.

Tasks:

- ☐ Containerize the service (multi-stage Dockerfile — Go builds to a static binary, so your final image can be tiny)
- ☐ Set up CI (GitHub Actions): run `go test -race ./...` and `go vet` on every push
- ☐ Set up CD: deploy on merge to main (Fly.io, Render, or GCP Cloud Run are all reasonable free/cheap options)
- ☐ Separate environments: at minimum, a staging deploy pointed at testnet and separate from anything you'd call "production"
- ☐ Secrets management: private keys and DB credentials via the platform's secret store — never committed, never in plain env files in the repo

Concepts this teaches: CI/CD fundamentals, environment separation, secrets hygiene — the last one matters more here than in almost any other kind of project, since this system quite literally holds private keys.

Phase 7 — Portfolio Polish

Goal: turn the finished system into something that actually gets you interviews.

Tasks:

- ☐ Update the README with an architecture diagram and a "decisions worth talking about" section (same format as your Stoxava pack — problem, options, what you chose, why)
- ☐ Record one video walking through the deposit-to-credit flow live, from a real testnet transaction to your Grafana dashboard lighting up
- ☐ Write down 3–5 things you'd do differently at higher scale (this is a genuinely great interview answer, and you can only give it honestly after having built the smaller version first)

What you'll be able to say when this is done

Not "I built a crypto wallet" — but: *I built a custodial wallet system that handles real on-chain deposits and withdrawals, processes them through a fault-tolerant queue-based pipeline with distributed locking, and is fully observable and deployed with CI/CD.* That sentence is true phase-by-phase as you complete each one — which means you have a legitimate, escalating story to tell in an interview at any point in this roadmap, not just at the very end.